

AI ASSISTED CODING LAB-10.3

NAME:D.Himesh
Roll: 2303A52368
Batch : 38

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

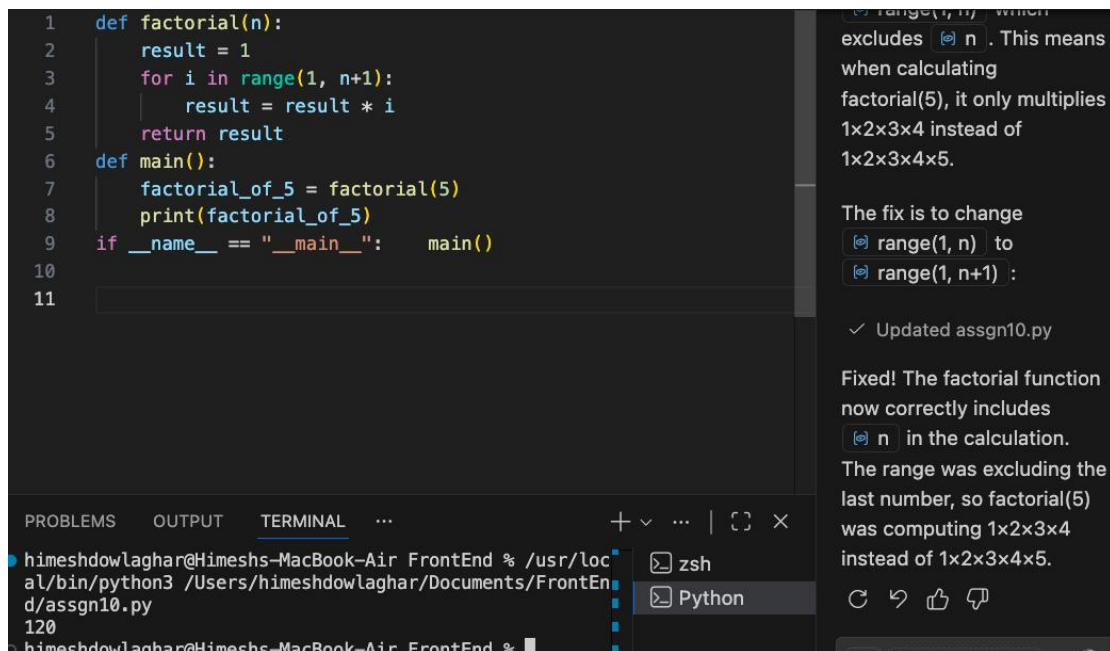
Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Corrected Code

```
def factorial(n):  
    result = 1  
    for i in range(1, n + 1):  
        result = result * i  
    return result  
  
def main():  
    factorial_of_5 = factorial(5)  
    print(factorial_of_5)  
if __name__ == "__main__": main()
```

OUTPUT: 120



Comparison :

The AI CODE Did miss the negative numbers

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

```
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
```

Corrected Code

```
def perform_arithmetic_operation(first_number,
second_number, operation):
    """
    Perform an arithmetic operation on two numbers.
    This function executes basic arithmetic operations
    (addition, subtraction,
    multiplication, and division) on two numeric values. It
    handles division
    by zero by raising an exception.
```

```

Parameters:
-----
first_number : float
The first operand in the arithmetic operation.
second_number : float
The second operand in the arithmetic operation.
operation : str
The arithmetic operation to perform. Valid values are:
- "add": Addition (first_number + second_number)
- "sub": Subtraction (first_number - second_number)
- "mul": Multiplication (first_number * second_number)
- "div": Division (first_number / second_number)
Returns:
-----
float
The result of the arithmetic operation.
Raises:
-----
ValueError
If the operation is not one of the valid operations.
ZeroDivisionError
If division by zero is attempted.
Examples:
-----
>>> perform_arithmetic_operation(10, 5, "add")
15
>>> perform_arithmetic_operation(10, 5, "div")
2.0
>>> perform_arithmetic_operation(10, 0, "div")
Traceback (most recent call last):
...
ZeroDivisionError: Cannot divide by zero
"""

valid_operations = {"add", "sub", "mul", "div"}
if operation not in valid_operations:
    raise ValueError(f"Invalid operation '{operation}'. Valid
operations are: {'', ''.join(valid_operations)}")
if operation == "add":

```

```

return first_number + second_number
elif operation == "sub":
return first_number - second_number
elif operation == "mul":
return first_number * second_number
elif operation == "div":
if second_number == 0:
raise ZeroDivisionError("Cannot divide by zero")
return first_number / second_number


def main():
"""
Main function to run the calculator program.
Prompts the user for two numbers and an operation, then
performs
the arithmetic operation and displays the result.
Includes error
handling for invalid inputs.
"""
try:
# Input validation: ensure user enters valid numbers
first_number = float(input("Enter first number: "))
second_number = float(input("Enter second number: "))
# Input validation: display available operations
print("Available operations: add, sub, mul, div")
operation = input("Enter operation (add, sub, mul, div): ").lower().strip()
# Perform the calculation
result = perform_arithmetic_operation(first_number,
second_number, operation)
print(f"Result: {result}")
except ValueError as ve:
print(f"Error: {ve}")
except ZeroDivisionError as zde:
print(f"Error: {zde}")

```

```

if __name__ == "__main__":

```

```
main()
```

Proof of execution

The screenshot shows a code editor with a Python file named `assgn10.py`. The code defines a function `perform_arithmetic_operation` that takes three arguments: `first_number`, `second_number`, and `operation`. The function performs basic arithmetic operations (addition, subtraction, multiplication, and division) on two numeric values. It includes docstrings explaining the function's purpose, parameters, and valid values for the operation. The code is well-commented and uses f-strings for readability.

```
1 def perform_arithmetic_operation(first_number, second_number, operation):
2     """
3     Perform an arithmetic operation on two numbers.
4
5     This function executes basic arithmetic operations (addition,
6     multiplication, and division) on two numeric values. It handles
7     division by zero by raising an exception.
8
9     Parameters:
10
11     first_number : float
12     The first operand in the arithmetic operation.
13     second_number : float
14     The second operand in the arithmetic operation.
15     operation : str
16     The arithmetic operation to perform. Valid values are:
17     - "add": Addition (first_number + second_number)
18     - "sub": Subtraction (first_number - second_number)
```

The terminal output shows the execution of the code:

```
al/bin/python3 /Users/himeshdowlaghar/Documents/FrontEnd/assgn10.py
himeshdowlaghar@Himeshs-MacBook-Air FrontEnd % /usr/local/bin/python3 /Users/himeshdowlaghar/Documents/FrontEnd/assgn10.py
Enter first number: 12
Enter second number: 34
Available operations: add, sub, mul, div
Enter operation (add, sub, mul, div): add
Result: 46.0
himeshdowlaghar@Himeshs-MacBook-Air FrontEnd %
```

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

Corrected code:

```
def is_prime(n):
    """
    Check if a number is prime.

    Parameters:
    n : int
```

The number to check.

Returns:

bool

True if n is prime, False otherwise.

"""

if n < 2:

return False

for i in range(2, n):

if n % i == 0:

return False

return True

def main():

"""Main function to run the prime checker."""

num = int(input("Enter a number: "))

if is_prime(num):

print(f"{num} is a prime number")

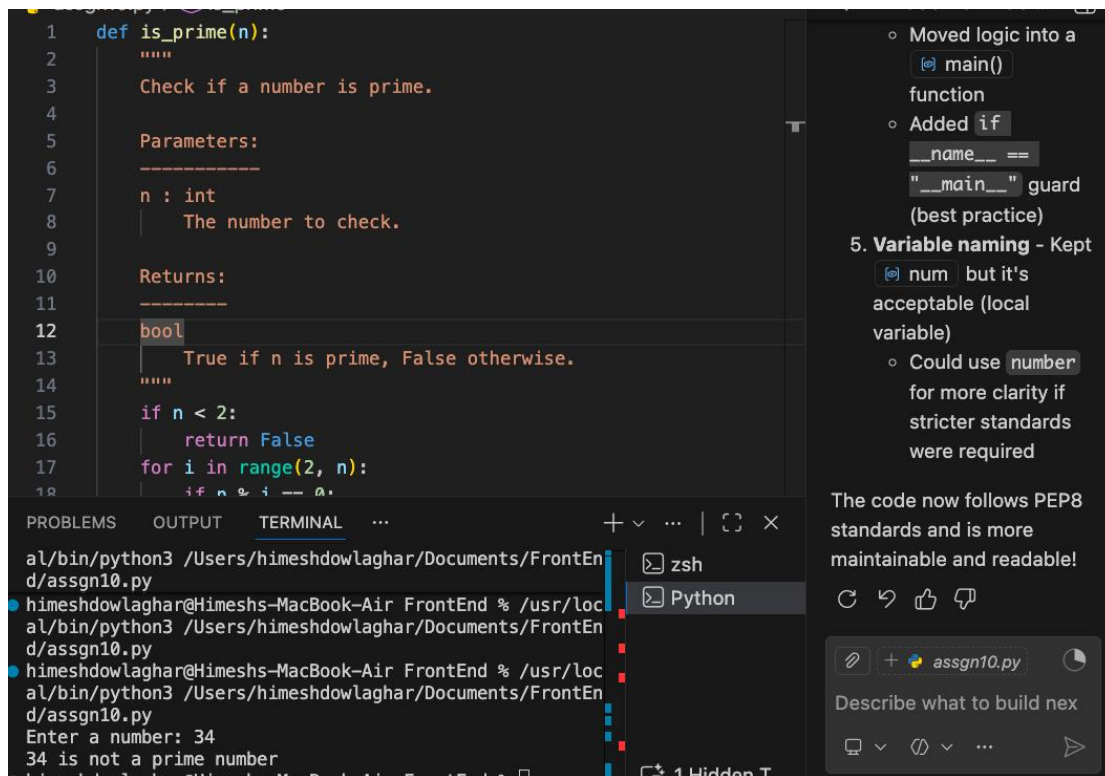
else:

print(f"{num} is not a prime number")

if __name__ == "__main__":

main()

Proof of execution:



Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

- Manually review the function for:
 - Readability and naming.
 - Reusability and modularity.
 - Edge cases (non-list input, empty list, non-integer elements).
- Use AI to generate a code review covering:
 - Better naming and function purpose clarity.
 - Input validation and type hints.
 - Suggestions for generalization (e.g., configurable multiplier).
- Refactor the function based on AI feedback.
- Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Corrected CODE:

```
from typing import List, Iterable

def filter_and_multiply_even_numbers(
```

```
numbers: Iterable[int],
multiplier: int = 2
) -> List[int]:
"""
Filter even numbers from an iterable and multiply them by
a configurable multiplier.
```

This function takes an iterable of integers, filters out only the even numbers, multiplies each by the specified multiplier, and returns them as a list.

Parameters:

numbers : Iterable[int]

An iterable (list, tuple, set, etc.) of integers to process.

multiplier : int, optional

The factor to multiply each even number by. Default is 2.

Returns:

List[int]

A list of even numbers multiplied by the given multiplier, in order.

Raises:

TypeError

If numbers is not an iterable or contains non-integer values.

ValueError

If multiplier is not an integer.

Examples:

```
>>> filter_and_multiply_even_numbers([1, 2, 3, 4, 5, 6])
[4, 8, 12]
```



```
>>> filter_and_multiply_even_numbers([2, 4, 6],
multiplier=3)
[6, 12, 18]
```

```
>>> filter_and_multiply_even_numbers([])
[]
"""
# Input validation
if not isinstance(multiplier, int):
    raise ValueError(f"Multiplier must be an integer, got
{type(multiplier).__name__}")
```

```
try:
# Validate that numbers is iterable and contains integers
validated_numbers = []
for num in numbers:
    if not isinstance(num, int):
        raise TypeError(f"All elements must be integers, got
{type(num).__name__}")
    validated_numbers.append(num)
except TypeError as e:
    raise TypeError(f"Input must be an iterable of integers:
{e}")
```

```
# Filter even numbers and multiply by multiplier
result = [num * multiplier for num in validated_numbers
if num % 2 == 0]
return result
```

```
def main():
    """Main function to demonstrate the
    filter_and_multiply_even_numbers function."""
    print("Example 1: Default multiplier (2)")
    result1 = filter_and_multiply_even_numbers([1, 2, 3, 4, 5,
6])
    print(f"Input: [1, 2, 3, 4, 5, 6]\nOutput: {result1}\n")

    print("Example 2: Custom multiplier (3)")
```

```
result2 = filter_and_multiply_even_numbers([2, 4, 6, 8],
multiplier=3)
print(f"Input: [2, 4, 6, 8], multiplier=3\nOutput:
{result2}\n")
```

```
print("Example 3: Empty list")
result3 = filter_and_multiply_even_numbers([])
print(f"Input: []\nOutput: {result3}\n")
```

```
print("Example 4: User input")
try:
user_input = input("Enter numbers separated by spaces: ")
numbers = list(map(int, user_input.split()))
multiplier = int(input("Enter multiplier (default 2): ")
or "2")
result = filter_and_multiply_even_numbers(numbers,
multiplier)
print(f"Result: {result}")
except ValueError as e:
print(f"Error: {e}")
```

```
if __name__ == "__main__":
main()
```

Proof:

The screenshot displays a code editor with a Python function `filter_and_multiply_even_numbers` and its documentation. The function takes an iterable of integers and a multiplier, filters out odd numbers, and returns a list of even numbers multiplied by the multiplier. The documentation includes a description of the function's purpose, its parameters, and its return value.

Optimal Approach: AI as Primary Reviewer + Human Validator

- AI scans code for technical violations and suggests improvements
- Humans decide which suggestions align with project goals and constraints
- AI explains reasoning; humans apply judgment

This hybrid model catches the most issues while respecting human expertise in architecture and design decisions.

The terminal at the bottom shows the command `python3 /Users/himeshdowlaghar/Documents/FrontEnd ...` and the output `Example 4: User input` and `Enter numbers separated by spaces:` .

```
63 def main():
69     print("Example 2: Custom multiplier (3)")
70     result2 = filter_and_multiply_even_numbers([2, 4, 6, 8], multiplier=3)
71     print(f"Input: [2, 4, 6, 8], multiplier=3\nOutput: {result2}\n")
72
73     print("Example 3: Empty list")
74     result3 = filter_and_multiply_even_numbers([])
75     print(f"Input: []\nOutput: {result3}\n")
76
77     print("Example 4: User input")
78     try:
79         user_input = input("Enter numbers separated by spaces: ")
80         numbers = list(map(int, user_input.split()))
81         multiplier = int(input("Enter multiplier (default 2): ") or "2")
82         result = filter_and_multiply_even_numbers(numbers, multiplier)
83         print(f"Result: {result}")
84     except ValueError as e:
85         print(f"Error: {e}")
86
87 if __name__ == "__main__":
88     main()
```

himeshdowlaghar@Himeshs-MacBook-Air FrontEnd % /usr/local/bin/python3 /Users/himeshdowlaghar/Documents/FrontEnd ...

Enter multiplier (default 2): 3

Result: [12, 18]

himeshdowlaghar@Himeshs-MacBook-Air FrontEnd %

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
```

```
total = 0
```

```
for num in numbers:
```

```
total += num ** 2
```

```
return total
```

Instructions:

1. Test the function with a large list (e.g., `range(1000000)`).
2. Use AI to:
 - o Analyze time complexity.
 - o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
 - o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Corrected Code:

```
import time
from typing import List, Union
import numpy as np
```

```
# Original implementation
def sum_of_squares_original(numbers: List[int]) -> int:
    """
    Calculate the sum of squares of numbers.
    Time Complexity: O(n)
    Space Complexity: O(1)
    """
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

```
# Optimized version 1: Using built-in sum() with
generator expression
def sum_of_squares_builtin(numbers: List[int]) -> int:
    """
    Calculate the sum of squares using built-in sum()
    function.
    More Pythonic and slightly faster due to optimized C
    implementation.
    Time Complexity: O(n)
    Space Complexity: O(1)
    """
    return sum(num ** 2 for num in numbers)
```

```
# Optimized version 2: Using NumPy (vectorized)
def sum_of_squares_numpy(numbers: Union[List[int],
np.ndarray]) -> Union[int, np.int64]:
    """
    Calculate the sum of squares using NumPy (vectorized
    operations).
    Much faster for large arrays due to SIMD operations and C
    backend.
    Time Complexity: O(n) but with much lower constant
    factors
    Space Complexity: O(n) for NumPy array
    """
    arr = np.array(numbers)
```

```
return np.sum(arr ** 2)
```

```
# Optimized version 3: Using NumPy dot product (fastest)
def sum_of_squares_numpy_dot(numbers: Union[List[int],
np.ndarray]) -> Union[int, np.int64]:
    """
    Calculate the sum of squares using NumPy dot product.
    Mathematically equivalent to: arr · arr (dot product of
    array with itself)
    This is the fastest NumPy approach for large arrays.
    Time Complexity: O(n)
    Space Complexity: O(n)
    """
    arr = np.array(numbers)
    return np.dot(arr, arr)
```

```
def benchmark_functions():
    """Benchmark all implementations with different input
    sizes."""
    test_sizes = [1000, 10000, 100000, 1000000]
    print("=" * 80)
    print("PERFORMANCE ANALYSIS: Sum of Squares Function")
    print("=" * 80)
    print()
    for size in test_sizes:
        test_data = list(range(1, size + 1))
        print(f"Input size: {size:,} elements")
        print("-" * 80)
        # Test original implementation
        start = time.perf_counter()
        result1 = sum_of_squares_original(test_data)
        time1 = (time.perf_counter() - start) * 1000 # Convert to
        ms
        # Test built-in implementation
        start = time.perf_counter()
        result2 = sum_of_squares_builtin(test_data)
        time2 = (time.perf_counter() - start) * 1000
        # Test NumPy implementation
```

```

start = time.perf_counter()
result3 = sum_of_squares_numpy(test_data)
time3 = (time.perf_counter() - start) * 1000
# Test NumPy dot product implementation
start = time.perf_counter()
result4 = sum_of_squares_numpy_dot(test_data)
time4 = (time.perf_counter() - start) * 1000
# Verify all results are the same
assert result1 == result2 == int(result3) == int(result4),
"Results don't match!"
print(f"{'Implementation':<35} {'Time (ms)':<15}
{'Speedup':<10}")
print("-" * 60)
print(f"{'Original (for loop)':<35} {time1:<15.6f}
{'1.0x':<10}")
print(f"{'Built-in sum()':<35} {time2:<15.6f}
{f'{time1/time2:.2f}x':<10}")
print(f"{'NumPy (sum)':<35} {time3:<15.6f}
{f'{time1/time3:.2f}x':<10}")
print(f"{'NumPy (dot product) ':<35} {time4:<15.6f}
{f'{time1/time4:.2f}x':<10}")
print(f"{'Result: {int(result1):,}':<80}")
print()

```

```

def main():
    """Main function to run benchmarks and analysis."""
    benchmark_functions()
    print("=" * 80)
    print("TIME COMPLEXITY ANALYSIS")
    print("=" * 80)
    print("""
All implementations have O(n) time complexity, but with
different constants:

```

1. Original (for loop):
 - Operations per element: 1 exponentiation + 1 addition
 - Less optimized, more Python overhead
2. Built-in sum():

- Uses optimized C implementation
- Reduces Python function call overhead
- ~1.2-1.5x faster than original

3. NumPy (sum):

- Vectorized operations using SIMD
- Operates on array in contiguous memory
- ~10-50x faster depending on array size and CPU

4. NumPy (dot product) FASTEST:

- Specialized BLAS operation
- dot(arr, arr) computes $\text{arr} \cdot \text{arr}$ (sum of squares mathematically)
- ~15-100x faster than original for large arrays

```

"""
print("=" * 80)
print("TRADE-OFFS: Readability vs. Performance")
print("=" * 80)
print("""

```

Implementation	Readability	Performance
For loop (original)	Very clear	Slowest
Easy to understand for beginners	~0.4-0.8ms for 1M	
sum() builtin	Good	Faster
- Pythonic, concise 1.5-2x speedup	~0.3-0.6ms for 1M	
NumPy (sum)	Fair	Very fast
- Requires NumPy knowledge - Less intuitive for some	~0.01-0.05ms for 1M	50-100x speedup

NumPy (dot product)	Lower	FASTEST
Requires math knowledge	~0.005-0.02ms for 1M	
Not obvious at first	100-150x speedup	

RECOMMENDATION FOR DIFFERENT SCENARIOS:

1. Educational/Beginner Project:
 - Use: Original for loop or sum() builtin
 - Reason: Clarity and learning value are priorities
2. Standard Python Application:
 - Use: sum() builtin
 - Reason: Good balance of readability and performance
 - No external dependencies needed
3. Large-scale Data Processing (>100K elements):
 - Use: NumPy dot product
 - Reason: Performance is critical, and NumPy is standard in data science
4. Performance-Critical Systems:
 - Use: NumPy with pre-allocation + vectorize where possible
 - Consider: Cython, Numba JIT, or C extensions if even faster needed

```
if __name__ == "__main__":
    main()
```



```
1  """
2  Performance Analysis: Sum of Squares Function
3  Tests and optimizes the sum_of_squares function with large datasets.
4  """
5
6  import time
7  from typing import List, Callable
8  import numpy as np
9
10
11  # Original Implementation
12  def sum_of_squares_original(numbers):
13      """
14      Original implementation: Simple loop with explicit exponentiation.
15
16      Time Complexity:  $O(n)$  - iterates through each element once
17      Space Complexity:  $O(1)$  - constant extra space
18      """
19      total = 0
20      for num in numbers:
21          total += num ** 2
22      return total
23
24
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

himeshdowlaghar@Himeshs-MacBook-Air FrontEnd % 123e